

COMS 4776 Lecture 16: Diffusion

Richard Zemel

Exam 2

Will cover all material from Lecture 10 on RNNs through Lecture 16 on VAEs, except:

- LSTMs
- Neural Turing Machines
- State-space models
- Re-parametrization trick

Closed book and notes. No phones allowed.

Office hours:

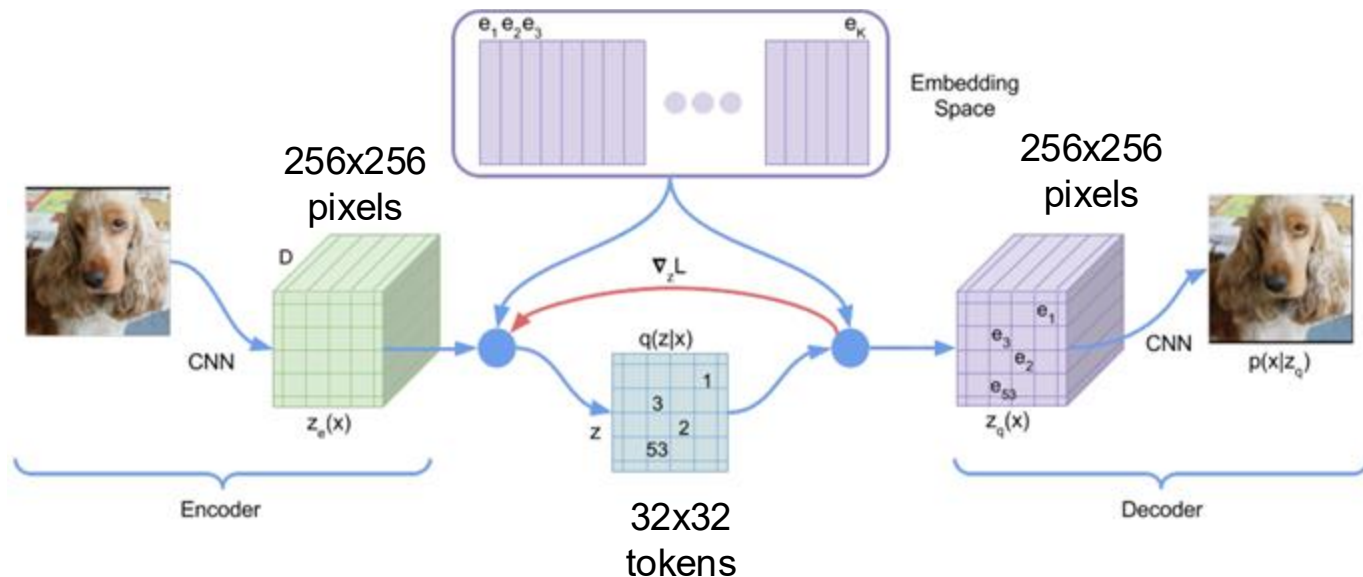
- Today at 4
- Zoom tomorrow 5-6
- Monday Dec 1 4-5, CEPSR 620

VQ-VAE: VAEs with Discrete Codes

Encoder: Tokenize a image into a map of tokens.

Decoder: Given a map of tokens, generate corresponding images

Objective: $-\log(p(x|q(x))) + ||\text{sg}[z_e(x)] - e||_2^2 + \beta ||z_e(x) - \text{sg}[e]||_2^2$



Diffusion Model: Intuition

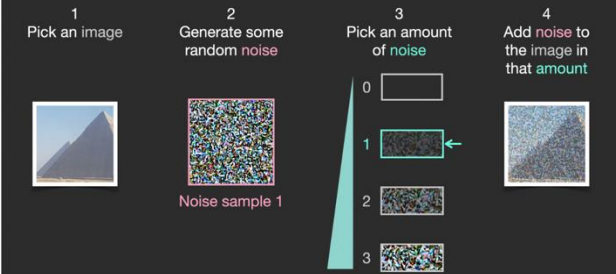
Key idea: random noise added to input example (image), in multiple steps

learning problem: learn to recover the data by reversing noising process

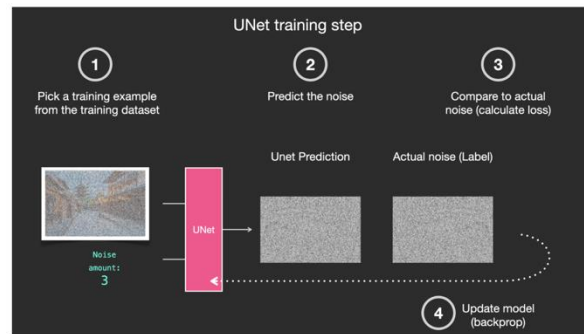
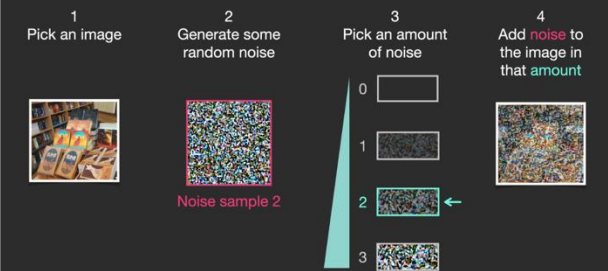
network predicts added noise → can predict denoised image

after training, run process in reverse: start with noise and gradually denoise, end up with image

Training examples are created by generating **noise** and adding an **amount** of it to the images in the training dataset (forward diffusion)



Generating a 2nd training example with a different image, **noise sample** and **noise amount** (forward diffusion)

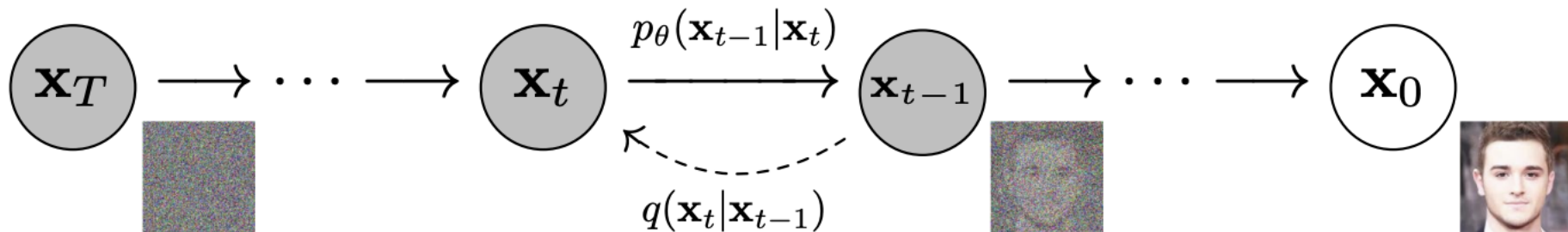


Diffusion Model: Details (optional)

Diffusion process: Markov chain gradually adds noise to data until signal is destroyed

Model: Parametrized chain trained using variational inference to produce data samples after finite time

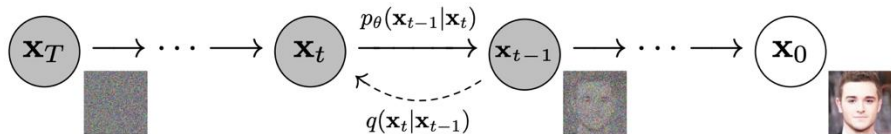
Intuition: replace encoder & decoder of VAE mapping to/from latent space with Markov Chains



$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) := \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}), \quad q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

$$p_\theta(\mathbf{x}_{0:T}) := p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t), \quad p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

Diffusion Model: Training (opt)



Objective:

$$\mathbb{E}[-\log p_{\theta}(\mathbf{x}_0)] \leq \mathbb{E}_q \left[-\log \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] = \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t \geq 1} \log \frac{p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right] =: L$$

If fix forward process variances to constants $\rightarrow$ approximate posterior has no parameters

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \bar{\boldsymbol{\mu}}_{\theta}(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}),$$

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t)\|^2 \right] + C$$

Reparametrization trick:

$$\text{sample } \mathbf{x}_{t-1} \sim p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

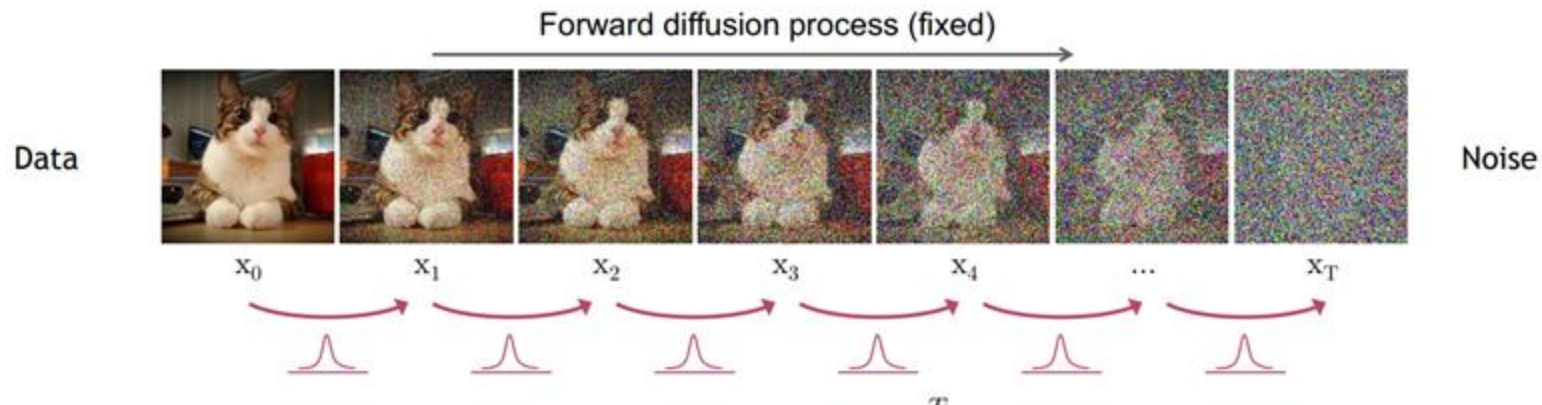
$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}, \text{ where } \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Train reverse process function approximator to predict noise:

$$\mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2 \right]$$

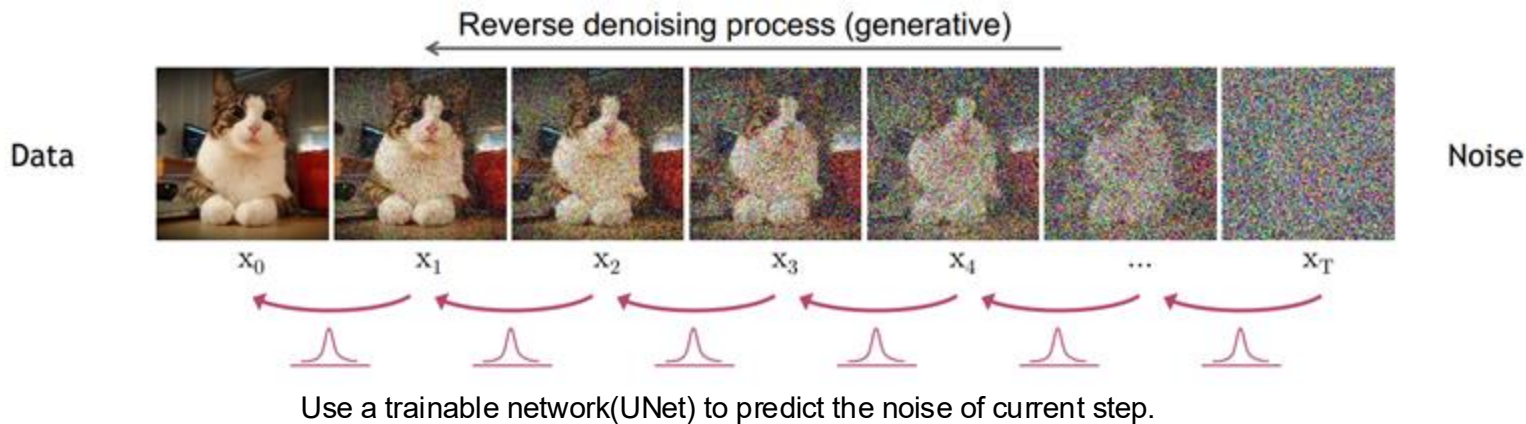
Diffusion Model

The formal definition of the forward process in T steps:



Diffusion Model

Formal definition of forward and reverse processes in T steps:



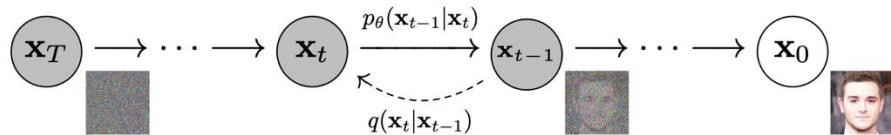
Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on
 $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$
 - 6: **until** converged
-

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 2: **for** $t = T, \dots, 1$ **do**
 - 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
 - 5: **end for**
 - 6: **return** $\mathbf{x}_0$
-

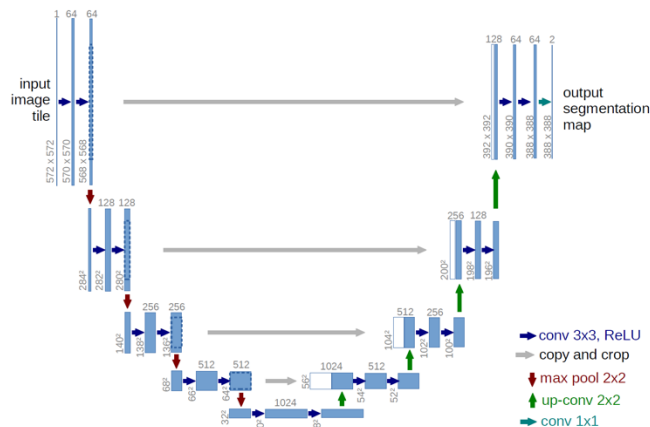
Diffusion Model: Details



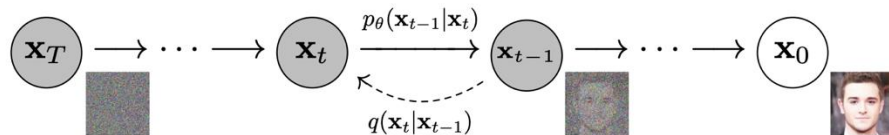
Produces state-of-the-art images, with high resolution

Model choices:

- variance schedule: typically time-dependent constants, increasing during forward process
- use same variances in reverse process --> training entails simply predicting posterior mean
- U-net (input and output same dimensionality)

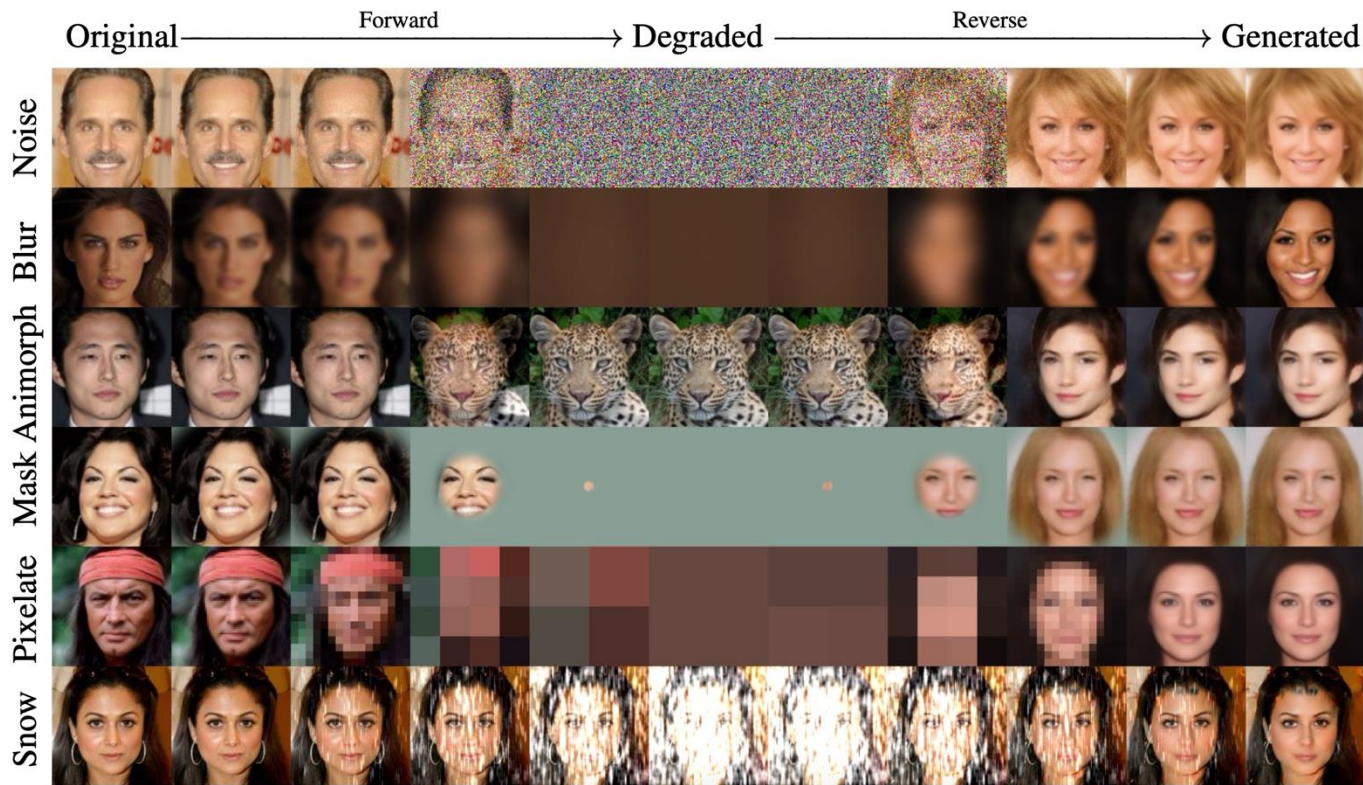


Diffusion Model: Summary



- Diffusion Model parameterized as Markov chain -- latent variables depend only on previous (or following) timestep.
- Transition distributions in Markov chain are Gaussian, where the forward process requires a variance schedule, and the reverse process parameters are learned.
- Diffusion process ensures that forward process converges on isotropic Gaussian for large T .
- Variance schedule can be fixed, or learned
- Diffusion Models allow for *any* architecture whose input and output dimensionality are the same to be used. Many implementations use U-Net-like architectures.
- Training objective is to maximize the likelihood of the training data -- minimize the variational upper bound of the negative log likelihood of the data.
- Almost all terms in the objective are KL Divergences due to Markov assumption; easy to calculate given Gaussian assumption
- Can use a simplified training objective to train a function which predicts the noise component of a given latent variable yields the best and most stable results.

Cold Diffusion

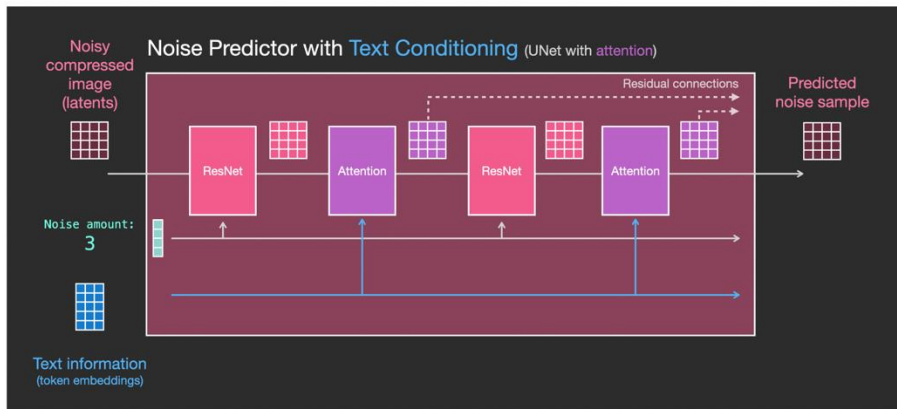


Bansal et al, 2022: “Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise”

Stable Diffusion

Two main enhancements to diffusion model:

1. Diffusion operates in latent space rather than pixel space
 - use autoencoder to map to/from latent space
 - VQ-VAE creates multiple tokens in latent space per image
2. Condition Markov chain on other information
 - embed text string / prompt
 - use Transformer to predict noise in diffusion model given text, noisy image embeddings



Stable Diffusion

